INTE 201: PROGRAMMING 3 (Structured Programming)

Name: Gueta, Ethan Ulrick D.
Instructor: Sir Victor James Ordoñez
Year and Section: DIT 2-3
Date Passed: October 25, 2026

Activities – Week 2 – Java Operators
Arithmetic • Relational • Logical • Assignment • Increment/Decrement • Precedence

BASIC

Four-Function Calculator

Read two numbers and apply every basic arithmetic operator to them, with clearly labeled output.

Requirements:

- Read two integers from the user (or declare them if input isn't required yet).
- Print the result of +, -, *, /, and % applied to the two numbers, each on its own labeled line.
- Use a comment to note which operation(s) behave differently for negative operands.

Program Link: https://www.jdoodle.com/ia/dcfg-aw2d-e8ej

Source Code:

```
import java.util.Scanner;

public class Week2_Basic_FourFunctionCalculator {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    /* Read two integers from the user (or declare them if input isn't required yet). */
    System.out.print("Enter first integer: ");
    int num1 = scanner.nextInt();

    System.out.print("Enter second integer: ");
    int num2 = scanner.nextInt();

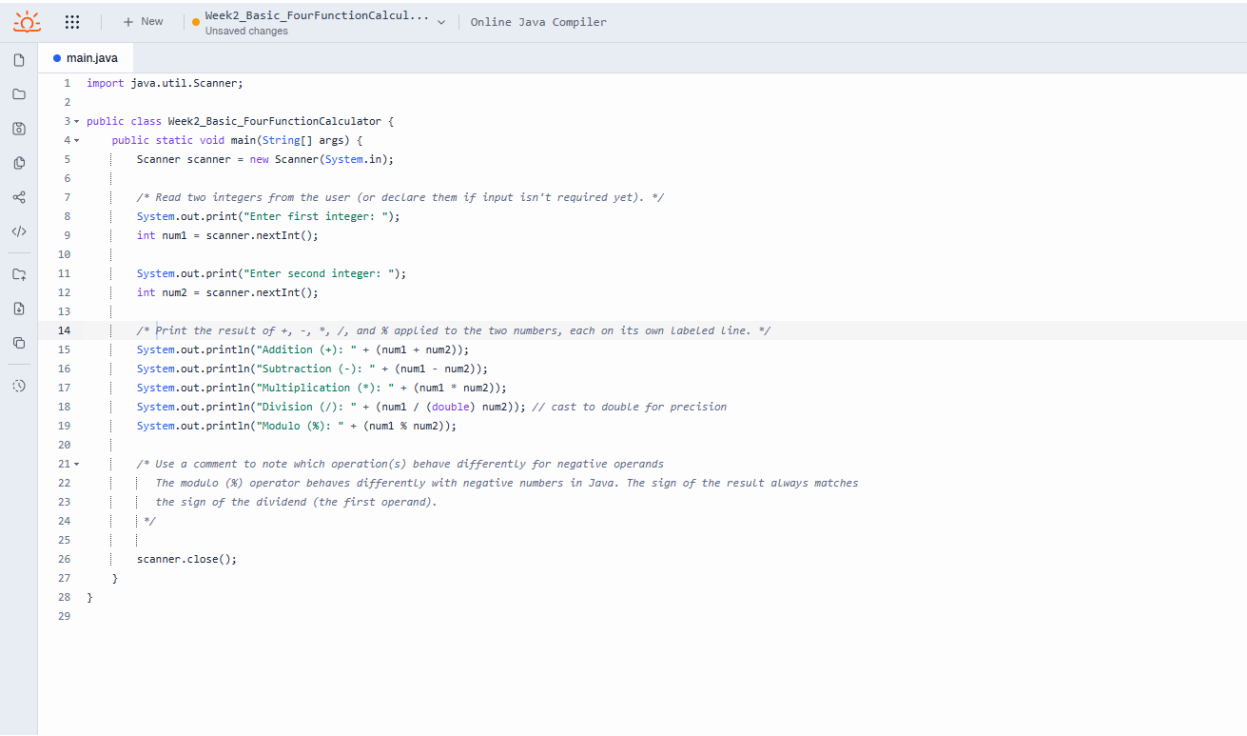
    /* Print the result of +, -, *, /, and % applied to the two numbers, each on its own labeled line. */
    System.out.println("Addition (+): " + (num1 + num2));
    System.out.println("Subtraction (-): " + (num1 - num2));
    System.out.println("Multiplication (*): " + (num1 * num2));
    System.out.println("Division (/): " + (num1 / (double) num2)); // cast to double for precision
    System.out.println("Modulo (%): " + (num1 % num2));

    /* Use a comment to note which operation(s) behave differently for negative operands
      The modulo (%) operator behaves differently with negative numbers in Java. The sign of the result always matches
      the sign of the dividend (the first operand).
     */
```

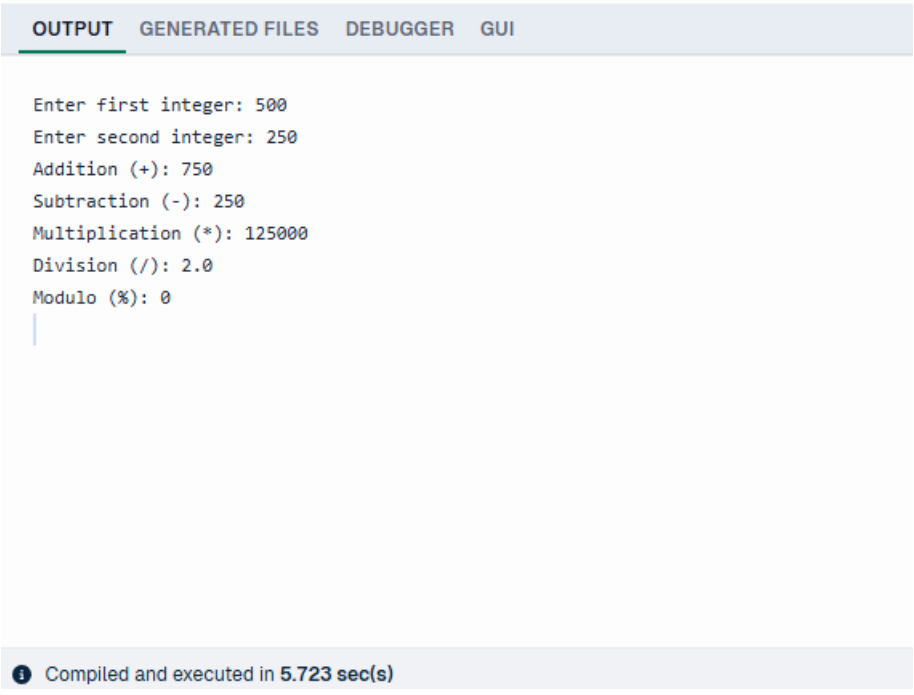
```
    scanner.close();
  }
}
```

Screenshots:

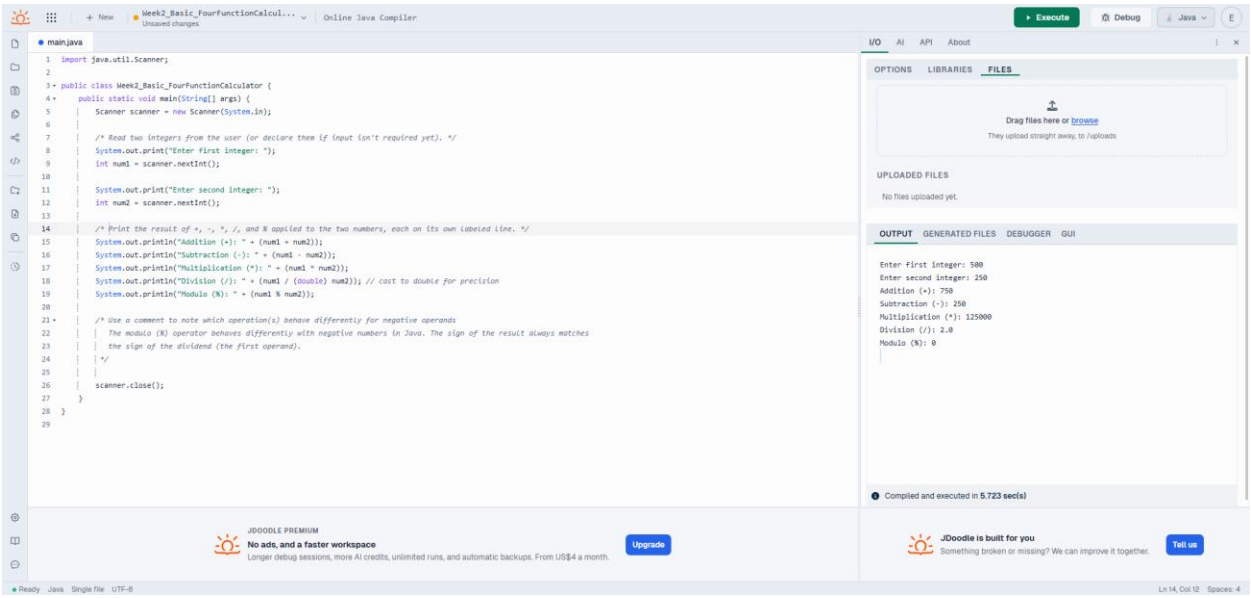
Input (Code):



Output:



Final Screenshot:



Compiler Used:

https://www.jdoodle.com/online-java-compiler

INTERMEDIATE

Grade Threshold Checker

Combine relational and logical operators to evaluate a multi-condition eligibility rule.

Requirements:

- Declare variables for a student's average grade and number of absences.
- Use && and || together (with parentheses) to determine if the student passes: average >= 75 AND absences <= 3, OR average >= 90 regardless of absences.
- Print a clear PASS/FAIL message along with the reasoning.

Program Link: https://www.jdoodle.com/ia/muy8-8pe4-t2xe

Source Code:

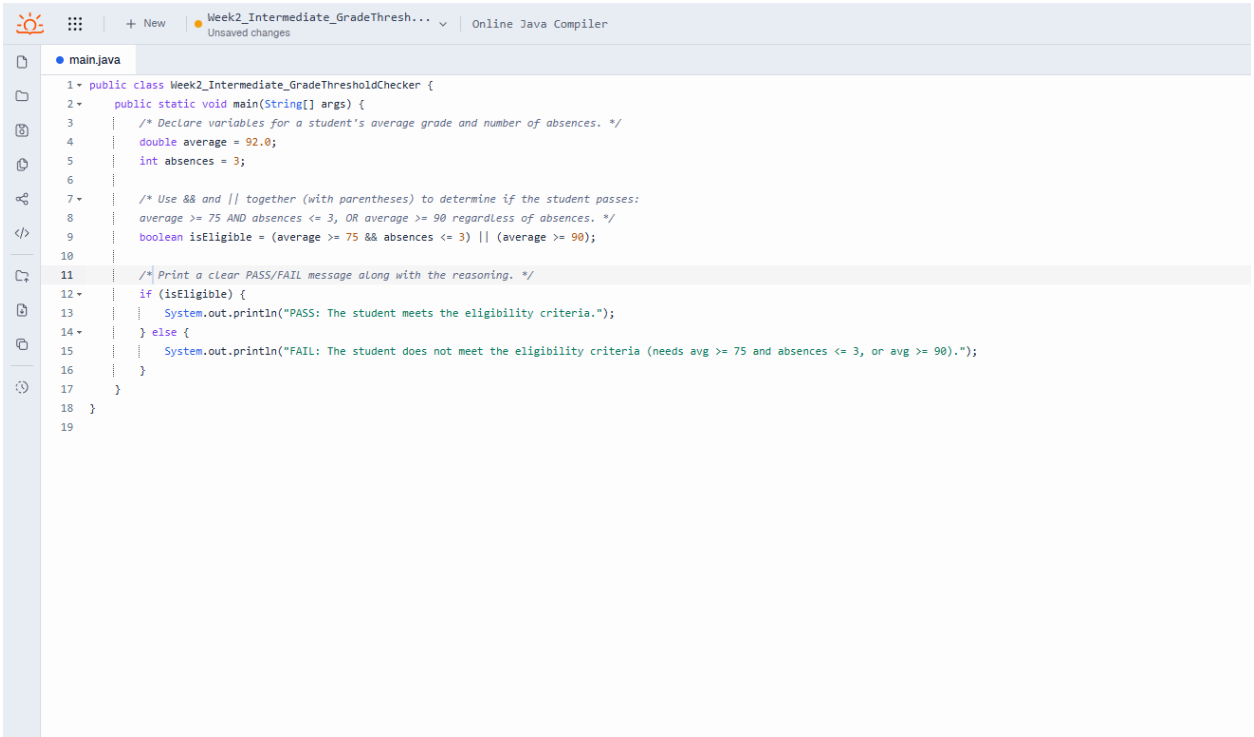
```
public class Week2_Intermediate_GradeThresholdChecker {
  public static void main(String[] args) {
    /* Declare variables for a student's average grade and number of absences. */
    double average = 92.0;
    int absences = 3;

    /* Use && and || together (with parentheses) to determine if the student passes:
    average >= 75 AND absences <= 3, OR average >= 90 regardless of absences. */
    boolean isEligible = (average >= 75 && absences <= 3) || (average >= 90);

    /* Print a clear PASS/FAIL message along with the reasoning. */
    if (isEligible) {
      System.out.println("PASS: The student meets the eligibility criteria.");
    } else {
      System.out.println("FAIL: The student does not meet the eligibility criteria (needs avg >= 75 and absences <= 3, or avg >= 90).");
    }
  }
}
```

Screenshots:

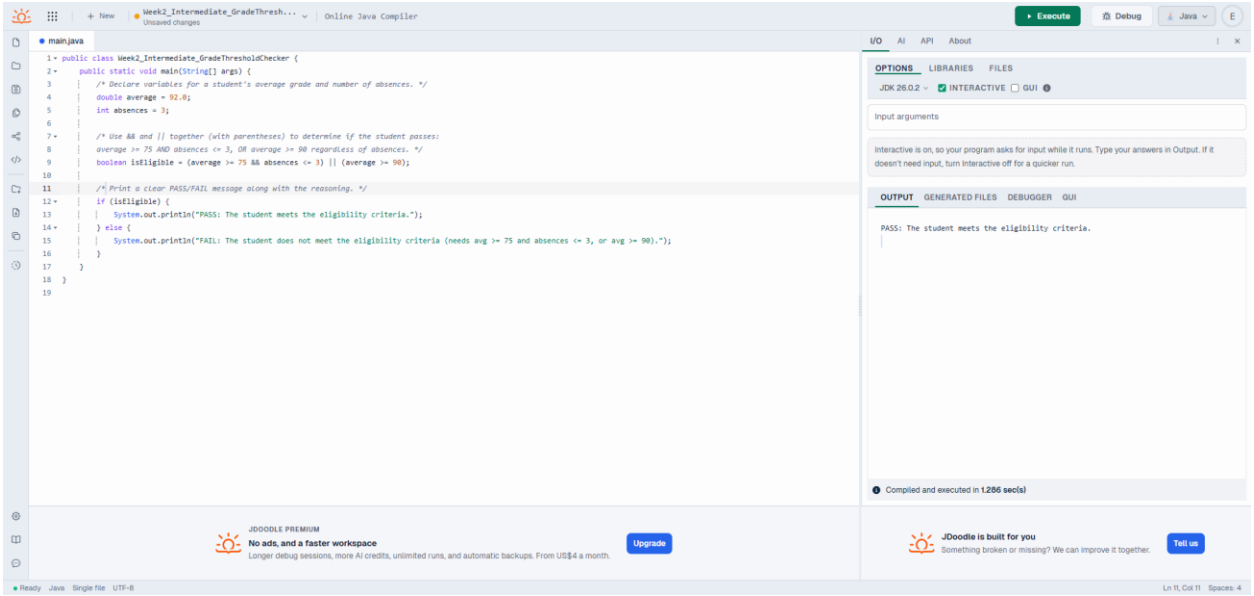
Input (Code):



Output:



Final Screenshot:



Compiler Used:

https://www.jdoodle.com/online-java-compiler

ADVANCED

Compound Growth & Increment Tracker

Use compound assignment operators to track a value changing over time, and prove the difference between prefix and postfix increment.

Requirements:

- Starting from an initial balance, apply += and *= across at least 3 separate statements to simulate growth over time, printing the balance after each step.
- Declare a counter variable and print the result of count++ inside a println(), then print count again immediately after.
- Repeat with ++count and explain in a comment exactly why the two printed values differ.

Program Link: https://www.jdoodle.com/ia/9egz-5ysq-gbur

Source Code:

```
public class Week2_Advanced_CompoundGrowthIncrementTracker {
  public static void main(String[] args) {

    double balance = 1000.0;
    System.out.println("Initial Balance: ₱" + balance);
    /* Starting from an initial balance, apply += and *= across at least 3 separate
    statements to simulate growth over time, printing the balance after each step. */

    balance += 500.0; // Statement 1
    System.out.println("Balance after adding ₱500: ₱" + balance);

    balance *= 1.05; // Statement 2 (5% interest)
    System.out.println("Balance after 5% interest: ₱" + balance);

    balance += 200.0; // Statement 3
    System.out.println("Balance after adding ₱200: ₱" + balance);

    System.out.println("\n--- Increment Tracker ---");
    int count = 5;

    /* Declare a counter variable and print the result of count++
    inside a println(), then print count again immediately after.*/
    System.out.println("Result of count++: " + (count++));
    System.out.println("Value of count immediately after: " + count);

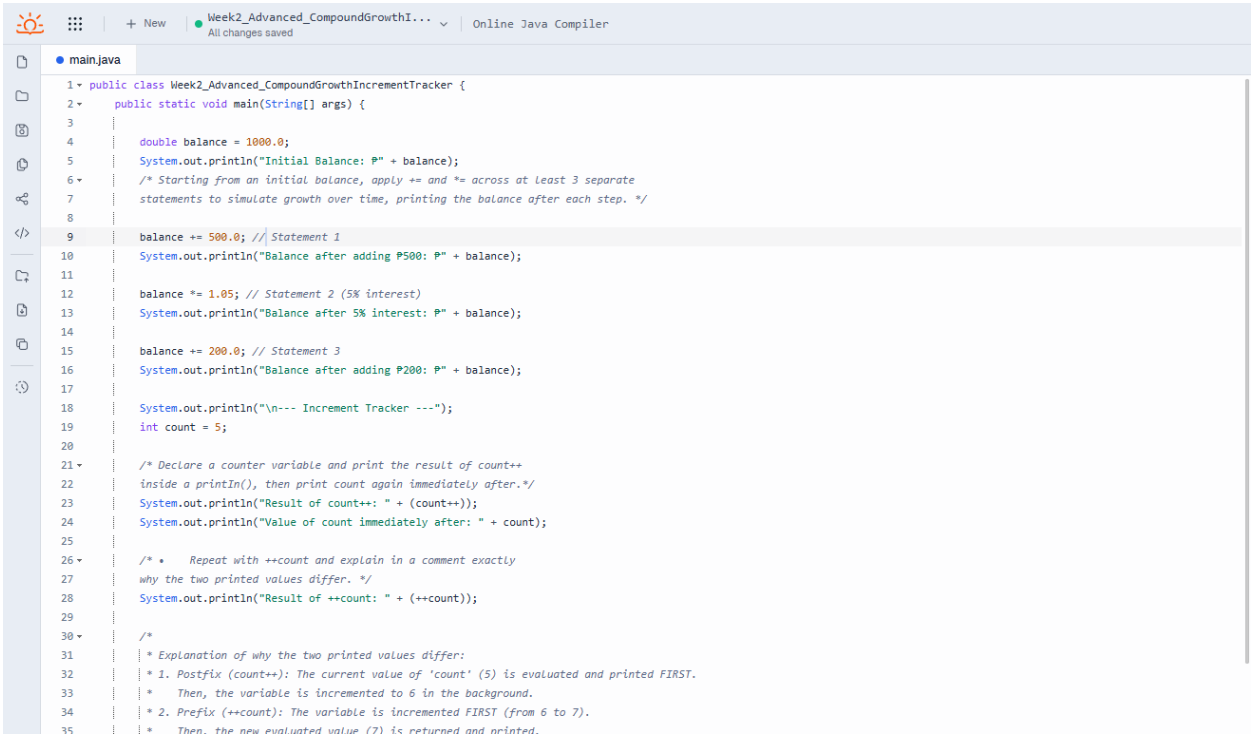
    /* •    Repeat with ++count and explain in a comment exactly
    why the two printed values differ. */
    System.out.println("Result of ++count: " + (++count));

    /*
     * Explanation of why the two printed values differ:
     * 1. Postfix (count++): The current value of 'count' (5) is evaluated and printed FIRST.
     *   Then, the variable is incremented to 6 in the background.
```

```
     * 2. Prefix (++count): The variable is incremented FIRST (from 6 to 7).
     *    Then, the new evaluated value (7) is returned and printed.
     */
  }
}
```

Screenshots:

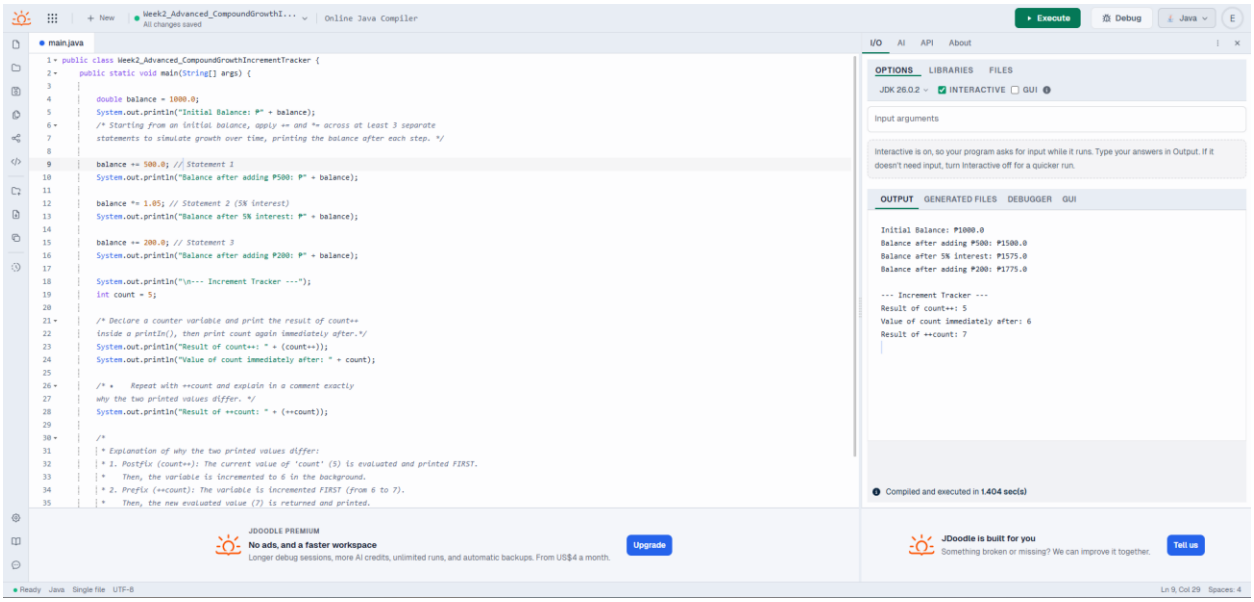
Input (Code):



Output:



Final Screenshot:



Compiler Used:

https://www.jdoodle.com/online-java-compiler

EXPERT

Precedence Puzzle Solver

Design and verify your own tricky expressions that combine multiple operator categories.

Requirements

- Write at least 3 expressions that each combine arithmetic, relational, logical, and/or increment/decrement operators.
- Before running each one, write your PREDICTED result in a comment directly above it.
- Print each actual result immediately after its prediction comment, so both can be compared.
- For any prediction that turns out wrong, add a follow-up comment explaining the precedence rule you missed.

Program Link: https://www.jdoodle.com/ia/n7tc-9z2m-989e

Source Code:

```
public class Week2_Expert_PrecedencePuzzleSolver {
  public static void main(String[] args) {
    int x = 5;
    int y = 10;

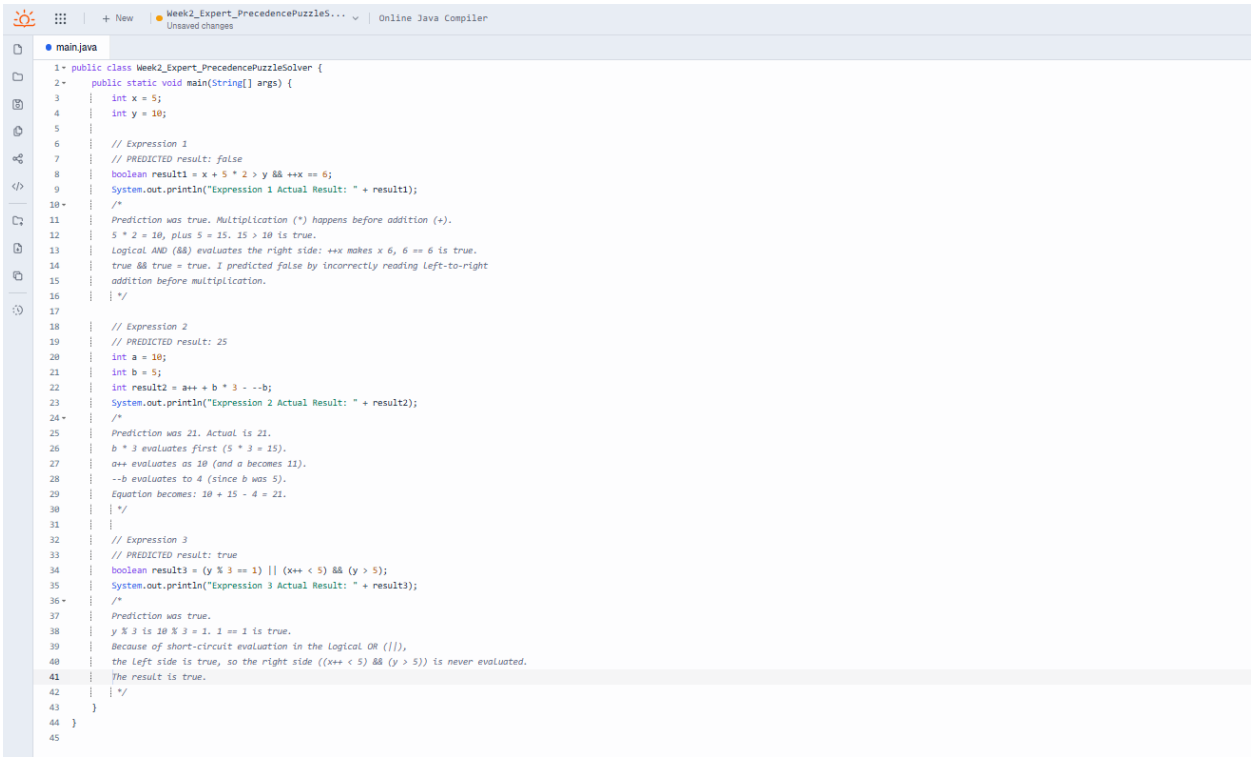
    // Expression 1
    // PREDICTED result: false
    boolean result1 = x + 5 * 2 > y && ++x == 6;
    System.out.println("Expression 1 Actual Result: " + result1);
    /*
    Prediction was true. Multiplication (*) happens before addition (+).
    5 * 2 = 10, plus 5 = 15. 15 > 10 is true.
    Logical AND (&&) evaluates the right side: ++x makes x 6, 6 == 6 is true.
    true && true = true. I predicted false by incorrectly reading left-to-right
    addition before multiplication.
     */

    // Expression 2
    // PREDICTED result: 25
    int a = 10;
    int b = 5;
    int result2 = a++ + b * 3 - --b;
    System.out.println("Expression 2 Actual Result: " + result2);
    /*
    Prediction was 21. Actual is 21.
    b * 3 evaluates first (5 * 3 = 15).
    a++ evaluates as 10 (and a becomes 11).
    --b evaluates to 4 (since b was 5).
    Equation becomes: 10 + 15 - 4 = 21.
     */

    // Expression 3
    // PREDICTED result: true
    boolean result3 = (y % 3 == 1) || (x++ < 5) && (y > 5);
    System.out.println("Expression 3 Actual Result: " + result3);
    /*
    Prediction was true.
    y % 3 is 10 % 3 = 1. 1 == 1 is true.
    Because of short-circuit evaluation in the logical OR (||),
    the left side is true, so the right side ((x++ < 5) && (y > 5)) is never evaluated.
    The result is true.
     */
  }
}
```

Screenshots:

Input (Code):



```java
public class Week2_Expert_PrecedencePuzzleSolver {
    public static void main(String[] args) {
        int x = 5;
        int y = 10;

        // Expression 1
        // PREDICTED result: false
        boolean result1 = x + 5 * 2 > y && ++x == 6;
        System.out.println("Expression 1 Actual Result: " + result1);
        /*
        Prediction was true. Multiplication (*) happens before addition (+).
        5 * 2 = 10, plus 5 = 15. 15 > 10 is true.
        Logical AND (&&) evaluates the right side: ++x makes x 6, 6 == 6 is true.
        true && true = true. I predicted false by incorrectly reading left-to-right
        addition before multiplication.
        */

        // Expression 2
        // PREDICTED result: 25
        int a = 10;
        int b = 5;
        int result2 = a++ + b * 3 - --b;
        System.out.println("Expression 2 Actual Result: " + result2);
        /*
        Prediction was 21. Actual is 21.
        b * 3 evaluates first (5 * 3 = 15).
        a++ evaluates as 10 (and a becomes 11).
        --b evaluates to 4 (since b was 5).
        Equation becomes: 10 + 15 - 4 = 21.
        */

        // Expression 3
        // PREDICTED result: true
        boolean result3 = (y % 3 == 1) || (x++ < 5) && (y > 5);
        System.out.println("Expression 3 Actual Result: " + result3);
        /*
        Prediction was true.
        y % 3 is 10 % 3 = 1. 1 == 1 is true.
        Because of short-circuit evaluation in the Logical OR (||),
        the left side is true, so the right side ((x++ < 5) && (y > 5)) is never evaluated.
        The result is true.
        */
    }
}
```

Output:

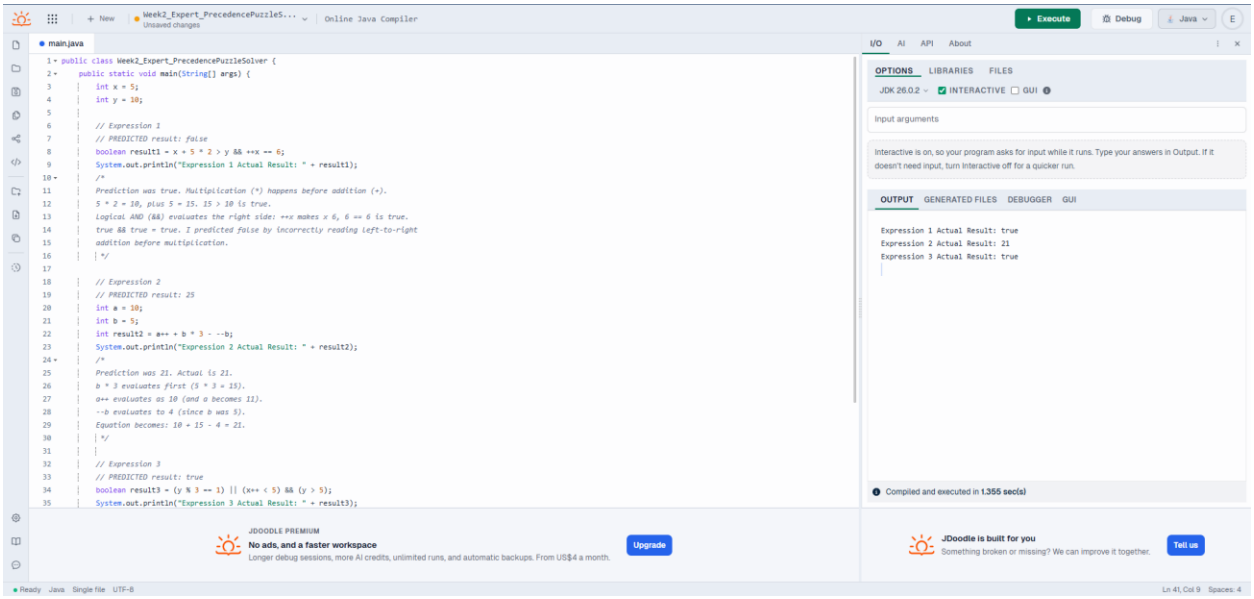


```
OUTPUT  GENERATED FILES  DEBUGGER  GUI

Expression 1 Actual Result: true
Expression 2 Actual Result: 21
Expression 3 Actual Result: true

Compiled and executed in 1.355 sec(s)
```

Final Screenshot:



Compiler Used:

https://www.jdoodle.com/online-java-compiler